RED & WHITE SKILL EDUCATION

DEEP LEARNING

PR 4

Project Definition

PROJECT A · CNN from Scratch — CIFAR-10 Image Classifier
PROJECT B · Teachable Machine — Export & Deploy in Python

Total Marks: 10

	Project A — CNN from Scratch	Project B — Teachable Machine
Approach	Build a CNN model entirely in TensorFlow/Keras code	Train a model visually on teachable machine.withgoogle.com, export it, and deploy in Python
Dataset	CIFAR-10 (built-in via keras.datasets)	Your own images / audio / body poses — collected live via webcam
Topics	CNN Architecture, Convolution, Padding, Strides, Pooling, MaxPooling, Flatten, Backprop in CNN, Data Augmentation, Transfer Learning (VGG16 / MobileNetV2), Fine-Tuning vs Feature Extraction	Teachable Machine Image Project, Audio Project, Pose Project — exported as TensorFlow SavedModel or .h5 — deployed and extended in Python
Output	Trained .h5 model + Jupyter Notebook + accuracy/loss plots	3 exported models (image / audio / pose) + 3 Python demo scripts + one combined application
Who should choose	Students comfortable with Python and Keras	Students who want a faster visual workflow or a creative real-world application focus
Marks	Same 10-mark scheme — same rubric applies to both	Same 10-mark scheme — same rubric applies to both

Choose ONE project only. You do not need to do both. Both projects earn the same 10 marks. Your choice must be declared at the top of your GitHub README.md.

PROJECT A — CNN FROM SCRATCH: CIFAR-10 IMAGE CLASSIFIER

Field	Detail
Project	Project A — CNN from Scratch
Subject	Deep Learning · Practical Report 4 (PR 4)
Dataset	CIFAR-10 — built into TensorFlow: tensorflow.keras.datasets.cifar10
Framework	TensorFlow 2.x / Keras, Python 3.x, Jupyter Notebook (DL_PR4A.ipynb)
Topics Covered	CNN History vs ANN, Convolution Operation, Padding & Strides, Pooling Layers, CNN Architecture, LeNet-5, Backpropagation in CNN, MaxPooling & Flatten, Data Augmentation, Transfer Learning (Feature Extraction + Fine-Tuning), Non-linear architectures
Project Duration	2 Days · 2 Hours per Day = 4 Hours total
Total Marks	10 (Component A: 5 · Component B: 3 · Component C: 2)

§ 1A · Dataset — CIFAR-10

CIFAR-10 (Canadian Institute For Advanced Research) is a benchmark image classification dataset containing 60,000 colour images across 10 classes. Each image is 32×32 pixels with 3 colour channels (RGB). The dataset is split into 50,000 training images and 10,000 test images, with exactly 6,000 images per class. It is built directly into TensorFlow/Keras — no download required. CIFAR-10 is the ideal CNN teaching dataset: small enough to train on a CPU within 10–30 minutes, yet complex enough that a simple MLP cannot achieve high accuracy — it requires convolutional feature extraction to do well.

Field	Detail
Loading	(X_train,y_train),(X_test,y_test) = keras.datasets.cifar10.load_data()
Training Set	50,000 images — 5,000 per class
Test Set	10,000 images — 1,000 per class
Image Shape	(32, 32, 3) — 32×32 pixels, RGB
Classes (10)	0=Airplane · 1=Automobile · 2=Bird · 3=Cat · 4=Deer · 5=Dog · 6=Frog · 7=Horse · 8=Ship · 9=Truck
Preprocessing	Normalise: X / 255.0. One-hot encode y: keras.utils.to_categorical(y, 10)
Baseline	Random classifier = 10% accuracy. Simple MLP ≈ 55%. A well-built CNN ≈ 80–88%

§ 2A · Project A Tasks — What to Do

Complete ALL tasks in DL_PR4A.ipynb. Each task builds on the previous. Add a Markdown cell at the start of each task: title + objective + expected output.

Task A1: Data Loading, EDA & Preprocessing

#	Action	Details / Configuration
A1.1	**Load CIFAR-10**	(X_train,y_train),(X_test,y_test) = keras.datasets.cifar10.load_data(). Print shapes: X_train (50000,32,32,3), X_test (10000,32,32,3). Print unique y values (0–9). Confirm dtype uint8.
A1.2	**Visualise sample images**	Plot a 5×10 grid — one column per class (10 classes), 5 rows. Title each column with the class name. Use plt.imshow (RGB, no cmap). Title the figure: 'CIFAR-10 — 5 Samples per Class'. Add a Markdown cell: note which classes are visually similar (cat/dog, automobile/truck) — these will be the hardest pairs for the CNN.
A1.3	**Pixel distribution & normalisation**	Plot a histogram of all pixel values from X_train.flatten() before normalisation (should be roughly uniform 0–255). Normalise: X_train = X_train.astype('float32') / 255.0; X_test = X_test.astype('float32') / 255.0. Plot histogram again after. Add a Markdown cell: WHY normalise — gradient descent converges faster when inputs are on [0,1]; large raw pixel values (up to 255) create proportionally large initial weight updates, destabilising early training.
A1.4	**One-hot encode labels**	y_train_ohe = keras.utils.to_categorical(y_train, 10). Confirm shape (50000,10). Add a Markdown cell: WHY one-hot for CNN output — the softmax output layer produces a probability distribution over 10 classes; categorical_crossentropy compares this distribution against the one-hot ground truth. Using integer labels would require sparse_categorical_crossentropy instead.
A1.5	**CNN vs ANN — Markdown**	Add a Markdown cell titled 'CNN vs ANN for Image Classification'. Cover: (1) ANN (MLP) flattens the image to a 1D vector — it loses all spatial relationships between pixels. (2) CNN applies a small learnable filter (kernel) that slides across the 2D image — each filter detects a local pattern regardless of where it appears (translation invariance). (3) CNN has far fewer parameters than an equivalent MLP because weights are shared across all spatial positions of the same filter. Example: a 3×3 filter on a 32×32 image shares the same 9 weights across all 30×30 valid positions — compared to 32×32=1,024 independent weights for the same spatial coverage in an MLP.

Task A2: Convolution Operation, Padding & Strides

#	Action	Details / Configuration
A2.1	**Manual convolution demo**	Before using Keras, demonstrate convolution manually on a single greyscale CIFAR image: (a) Extract one image, convert to greyscale: gray = np.mean(X_train[0], axis=2). (b) Define a 3×3 edge detection kernel: [[-1,-1,-1],[-1,8,-1],[-1,-1,-1]]. (c) Apply using scipy.signal.convolve2d(gray, kernel, mode='valid'). (d) Display: original greyscale + convolved output. Add a Markdown cell: the convolution operation computes a dot product between the kernel and each overlapping patch of the image — the kernel slides across the image one step at a time (stride), producing a new feature map. This exact

		operation is what CNN Conv2D layers perform on every channel simultaneously, with learnable kernel weights.
A2.2	**Padding — VALID vs SAME**	Using a test image and keras.layers.Conv2D directly: (a) VALID padding (no padding) — output is smaller: output_size = (input_size − kernel_size) / stride + 1. For 32×32 input, 3×3 kernel, stride 1: output = 30×30. (b) SAME padding (zero-pad the border) — output matches input size: 32×32. Build two minimal test models (Input → Conv2D(8, 3, padding) → Lambda(shape)) and print the output shape. Display the zero-padded image using np.pad. Add a Markdown cell: SAME padding is used in most CNN architectures to allow stacking many conv layers without shrinking the spatial dimensions too quickly.
A2.3	**Strides effect**	Build two test models: (a) stride=1 — output 30×30 (with VALID) or 32×32 (SAME). (b) stride=2 — output is halved: 16×16. Print both output shapes. Add a Markdown cell: stride controls how many pixels the filter moves at each step. stride=1 gives dense feature maps; stride=2 downsamples the spatial dimensions by 2× — an alternative to pooling that is learned rather than fixed. YOLOv8 uses strided convolutions instead of max-pooling.
A2.4	**Build and visualise feature maps**	Build a single Conv2D layer: model_vis = keras.Sequential([keras.layers.Conv2D(16, (3,3), activation='relu', padding='same', input_shape=(32,32,3))]). Pass one image and extract the 16 feature maps (output shape: 1×32×32×16). Display all 16 feature maps in a 4×4 grid. Add a Markdown cell: each feature map shows what one learnable filter responds to — early filters detect edges, gradients, and colours; deeper filters detect progressively complex patterns (textures, shapes, object parts).

Task A3: Pooling, MaxPooling, Flatten & CNN Architecture

#	Action	Details / Configuration
A3.1	**MaxPooling vs AveragePooling**	Append a MaxPooling2D(2,2) layer to the Task A2.4 visualisation model. Display the 16 feature maps before and after MaxPooling (they should be halved: 32×32 → 16×16). Repeat with AveragePooling2D(2,2). Add a Markdown cell: (1) MaxPooling2D(2,2) takes the maximum value from each 2×2 patch — retains the strongest activation (most prominent feature); gives slight translation invariance. (2) AveragePooling2D takes the mean — smoother, preserves more spatial information; generally less sharp. (3) In modern CNNs, GlobalAveragePooling2D (gap) is often preferred over Flatten — it averages each feature map to a single value, dramatically reducing parameters.
A3.2	**Flatten layer**	Add Flatten() after MaxPooling in the visualisation model. Print the output shape before and after Flatten: e.g., (None,16,16,16) → (None,4096). Add a Markdown cell: Flatten converts the 3D tensor (H, W, C) to a 1D vector, connecting it to a Dense classification head. The flattened size determines the number of parameters in the first Dense layer — this is why reducing spatial dimensions before Flatten is critical for keeping parameter counts manageable.
A3.3	**LeNet-5 architecture — implement from specification**	Build LeNet-5 exactly as originally described (Lecun et al., 1998) adapted for CIFAR-10 (32×32×3 input, 10 classes): Input(32,32,3) → Conv2D(6,(5,5),tanh) → AveragePooling2D(2,2) → Conv2D(16,(5,5),tanh) → AveragePooling2D(2,2) → Flatten → Dense(120,tanh) → Dense(84,tanh) → Dense(10,softmax). Print model.summary(). Count parameters manually for at least two layers. Add a Markdown cell: LeNet-5 (1998) was the first successful CNN — used to read handwritten US postal codes and cheques. Its architecture established the Conv→Pool→Conv→Pool→Flatten→Dense→Dense

		pattern still used today. The main differences from modern CNNs: it uses tanh (not ReLU), AveragePooling (not MaxPooling), and 5×5 filters (modern nets prefer stacked 3×3).
A3.4	**Build custom CNN baseline for CIFAR-10**	Build a modern CNN: Input(32,32,3) → Conv2D(32,(3,3),relu,SAME) → Conv2D(32,(3,3),relu,SAME) → MaxPooling2D(2,2) → Conv2D(64,(3,3),relu,SAME) → Conv2D(64,(3,3),relu,SAME) → MaxPooling2D(2,2) → Conv2D(128,(3,3),relu,SAME) → GlobalAveragePooling2D → Dense(256,relu) → Dense(10,softmax). Print model.summary(). Verify parameter count < 2M. Add a Markdown cell: WHY stacking two conv layers before each pooling layer — the first conv detects simple features (edges); the second composes them into slightly more complex patterns before the spatial resolution is reduced. This gives the network a richer feature hierarchy at each scale.

Task A4: Backpropagation in CNN & Training

#	Action	Details / Configuration
A4.1	**Backpropagation in CNN — Markdown**	Add a detailed Markdown cell: (1) Forward pass in CNN: input image → convolve with learnable kernels → activation → pool → repeat → flatten → dense → softmax → loss. (2) Backward pass — gradients flow backwards: ∂Loss/∂weights for Dense layers is identical to MLP backprop. ∂Loss/∂Conv weights requires computing the gradient of the loss w.r.t. each element of each kernel — this is done by correlating the upstream gradient (from pooling) with the local input patch. (3) Gradient through MaxPooling: only the position that produced the maximum value in the forward pass receives the full gradient; all other positions get zero — this is called the 'max location switch' or 'max index mask'. (4) Gradient through AveragePooling: the upstream gradient is divided equally among all positions in the pooling window. (5) Gradient through Flatten is trivial — it simply reshapes the 1D gradient back to 3D.
A4.2	**Compile and train the CNN**	model_cnn.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy']). from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau. es = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True). rlrop = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5, min_lr=1e-6, verbose=1). history = model_cnn.fit(X_train, y_train_ohe, epochs=100, batch_size=64, validation_split=0.1, callbacks=[es, rlrop]). Add a Markdown cell: ReduceLROnPlateau halves the learning rate when val_loss stops improving — allows the model to converge from a coarser search (high lr) to fine-tuning (low lr) without manual intervention.
A4.3	**Plot training curves**	Create a 1×2 subplot: (left) training vs validation loss; (right) training vs validation accuracy. Mark the epoch where EarlyStopping triggered with a vertical dashed red line. Mark the epochs where ReduceLROnPlateau fired with vertical grey dashed lines. Title: 'CNN Training Curves — CIFAR-10'. Add annotations showing the final train and val accuracy values.
A4.4	**Evaluate and analyse**	model_cnn.evaluate(X_test, y_test_ohe). Print test accuracy. Plot confusion matrix (10×10, class names on both axes). Identify the top 3 most-confused class pairs. Display 10 misclassified images (2×5 grid) with true and predicted labels. Add a Markdown cell: why does the CNN confuse cat/dog and automobile/truck more than airplane/ship? Relate this to visual similarity at 32×32 resolution — small, low-resolution images compress visual detail that distinguishes similar classes.

Task A5: Data Augmentation

#	Action	Details / Configuration
A5.1	WHY data augmentation — Markdown	Add a Markdown cell: a CNN with 2M parameters trained on 50,000 images risks memorising the training set. Data augmentation artificially expands the effective training set by applying random transformations to each image at training time — the model sees a different version of each image every epoch. This acts as a strong regulariser, reducing overfitting without collecting more data. It is especially important for small datasets (< 100K images) and is standard practice in production computer vision.
A5.2	Apply augmentation using ImageDataGenerator	from tensorflow.keras.preprocessing.image import ImageDataGenerator. datagen = ImageDataGenerator(rotation_range=15, width_shift_range=0.1, height_shift_range=0.1, horizontal_flip=True, zoom_range=0.1, fill_mode='nearest'). datagen.fit(X_train). Visualise the effect: take one training image and generate 9 augmented versions using datagen.flow(img[np.newaxis,...], batch_size=1). Plot the original + 9 augmented versions in a 2×5 grid. Title: 'Data Augmentation — 9 Random Transformations of One CIFAR-10 Image'.
A5.3	Retrain CNN with augmentation	Build the same CNN architecture as Task A3.4. Train using datagen.flow(X_train, y_train_ohe, batch_size=64) instead of raw arrays. Use the same callbacks (EarlyStopping, ReduceLROnPlateau). Store history_aug. Plot training curves comparing: model_cnn (no augmentation) vs model_aug (with augmentation) on the same 1×2 figure — 4 lines total. Add a Markdown cell: the augmented model should show a smaller gap between training and validation accuracy — indicating less overfitting. Training accuracy may be slightly lower (because each batch is harder) but test accuracy should be higher.
A5.4	Modern augmentation with Keras layers	Build a second augmented model using Keras preprocessing layers (preferred over ImageDataGenerator in TF 2.x): data_augmentation = keras.Sequential([keras.layers.RandomFlip('horizontal'), keras.layers.RandomRotation(0.1), keras.layers.RandomZoom(0.1), keras.layers.RandomTranslation(0.1,0.1)]). Prepend this to the CNN model. The augmentation layers are active during training and inactive during inference — no separate datagen.flow() needed. Train and compare test accuracy with the ImageDataGenerator approach.

Task A6: Transfer Learning — Feature Extraction & Fine-Tuning

#	Action	Details / Configuration
A6.1	WHY Transfer Learning — Markdown	Add a Markdown cell: training a deep CNN from scratch on CIFAR-10 takes hours and requires substantial data. Transfer Learning uses a model pre-trained on a much larger dataset (ImageNet — 14M images, 1,000 classes) and adapts it to a new task. The pre-trained model has already learned a hierarchy of visual features — edges, textures, shapes, object parts — that are useful across many vision tasks. Two strategies: (1) Feature Extraction — freeze all pre-trained weights, add a new classification head, train only the head. Fast (few minutes) but less accurate. (2) Fine-Tuning — unfreeze the top N layers of the base model and train them alongside the new head at a very low learning rate. Slower but significantly more accurate.
A6.2	Feature Extraction with MobileNetV2	base_model = keras.applications.MobileNetV2(input_shape=(96,96,3), include_top=False, weights='imagenet'). base_model.trainable = False. Note: CIFAR-10 images are 32×32 — too small for MobileNetV2 (designed for ≥ 96×96). Resize training data: X_train_r =

		tf.image.resize(X_train, (96,96)). Build: inputs = keras.Input(shape=(96,96,3)) → x = base_model(inputs, training=False) → x = keras.layers.GlobalAveragePooling2D()(x) → x = keras.layers.Dense(128, relu)(x) → outputs = keras.layers.Dense(10, softmax)(x). Print total vs trainable parameters — the frozen base should show most parameters as non-trainable.
A6.3	**Train feature extraction model**	model_fe.compile(optimizer=keras.optimizers.Adam(0.001), loss='categorical_crossentropy', metrics=['accuracy']). history_fe = model_fe.fit(X_train_r, y_train_ohe, epochs=20, batch_size=64, validation_split=0.1). Evaluate on resized test set. Record accuracy. Feature extraction should outperform the scratch CNN from Task A4 significantly, having been trained for only 20 epochs.
A6.4	**Fine-Tuning**	Unfreeze the top 30 layers of MobileNetV2: base_model.trainable = True. for layer in base_model.layers[:-30]: layer.trainable = False. Recompile at a very low learning rate: model_fe.compile(optimizer=keras.optimizers.Adam(1e-5), loss='categorical_crossentropy', metrics=['accuracy']). history_ft = model_fe.fit(X_train_r, y_train_ohe, epochs=10, batch_size=32, validation_split=0.1). Plot training curves: Feature Extraction history + Fine-Tuning history concatenated (mark the fine-tuning start with a vertical dashed line). Add a Markdown cell: WHY use a low lr for fine-tuning — the pre-trained weights are already good; a large lr would destroy them. Fine-tuning only nudges them toward the new task.
A6.5	**Full results comparison table**	Build a pandas DataFrame: Model | Architecture | Augmentation | Transfer Learning | Test Accuracy | Parameters. Rows: LeNet-5 (Task A3.3), CNN Scratch No Aug (Task A4), CNN Scratch + Aug (Task A5.3), CNN + Keras Aug Layers (Task A5.4), MobileNetV2 Feature Extraction (Task A6.3), MobileNetV2 Fine-Tuned (Task A6.4). Display with highlight_max on Test Accuracy. Add a Markdown cell: summarise the accuracy progression and the effort/accuracy trade-off of each approach.
A6.6	**Non-linear architecture — Residual connection**	Build a minimal residual block to demonstrate non-linear CNN architecture: inputs → Conv2D(64,1×1,SAME) [shortcut] → add with [Conv2D(64,3×3,relu,SAME) → Conv2D(64,3×3,relu,SAME)] → Activation(relu). Wrap in a Functional API model. Add a Markdown cell: non-linear (non-sequential) architectures allow skip connections — the gradient can bypass problematic layers during backprop, solving the vanishing gradient problem in very deep networks. This is the core innovation of ResNet (He et al., 2016).

§ 3A · Marking Scheme — Project A (10 Marks)

Total Marks: 10 — Component A (5) + Component B (3) + Component C (2).

Component	Marks	Weight	Deliverable
A. Technical Completion (DL_PR4A.ipynb)	5 Marks	50%	Notebook — all 6 task groups, Restart & Run All clean
B. Recorded Video Explanation (Face + Screen)	3 Marks	30%	5–10 min MP4/MOV: face + screen, verbal concept explanations
C. GitHub Repository + README.md	2 Marks	20%	Public repo with .ipynb + .html + plots + README + video link
TOTAL	**10 Marks**	**100%**	

Component A: Technical Completion (5 Marks)

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
A1	**0.5**	CIFAR-10 loaded; 5×10 sample grid; pixel histogram before/after normalisation; one-hot encoding confirmed; CNN vs ANN Markdown with parameter sharing explanation	Dataset loaded and normalised; sample grid; partial Markdown	Dataset loaded; no visualisation or Markdown	Dataset connected; no preprocessing	Not done
A2	**0.5**	Manual convolution with scipy on greyscale image; VALID vs SAME output shapes printed with formula; stride comparison (stride=1 vs stride=2 shapes); 16 feature maps from single Conv2D layer displayed in 4×4 grid	VALID vs SAME demonstrated; feature maps shown; manual conv or stride missing	Feature maps only; no VALID/SAME/stride comparison	Conv2D model built; no feature map visualisation	No convolution experiments

A3	**1.0**	MaxPooling vs AveragePooling feature maps before/after shown; Flatten output shape before/after; LeNet-5 built exactly to spec with parameter count verified; custom CNN (VGGNet-style) built with < 2M params and summary printed; WHY stacked conv layers explained	MaxPooling + Flatten + LeNet-5 built; custom CNN built; minor gap (no AveragePooling comparison or partial LeNet param count)	LeNet-5 and custom CNN built; no pooling comparison or Flatten explanation	One of LeNet-5 or custom CNN only	No architecture tasks
A4	**1.0**	Backprop in CNN Markdown: forward pass, gradient through MaxPool (max location switch), gradient through AveragePool, gradient through Flatten — all explained; model compiled with Adam + CCE; EarlyStopping + ReduceLROnPlateau callbacks; training curves with stopping epoch and LR reduction epochs annotated; confusion matrix; 10 misclassified samples grid	Model trained with callbacks; curves plotted; confusion matrix; backprop Markdown partially complete	Model trained; basic accuracy only; no callbacks; no Markdown	Model compiles but training incomplete	No training
A5+A6	**2.0**	ImageDataGenerator augmentation visualised (9 transforms of one image); CNN retrained with augmentation; comparison plot (no-aug vs aug, 4 lines); Keras aug layers	Augmentation + Feature Extraction + Fine-Tuning all functional; results table present; residual block or 6-model table partially incomplete	Augmentation + Feature Extraction; no Fine-Tuning or results table	Augmentation only; no Transfer Learning	None

		model; MobileNetV2 Feature Extraction (trainable params shown, 20 epochs); Fine-Tuning (top-30 layers unfrozen, lr=1e-5, concatenated history plot with fine-tune start line); full 6-model results table; residual connection block in Functional API; Restart & Run All clean				

Subtotal A: 0.5 + 0.5 + 1.0 + 1.0 + 2.0 = 5.0 Marks

Component B: Recorded Video — Face + Screen (3 Marks)

Record a video showing your face (webcam PiP) and Jupyter Notebook screen. Explain concepts verbally — do not just run cells silently.

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
B1	**1.0**	Clear audio and video throughout; face visible via PiP; notebook outputs and plots visible; all 6 task groups demonstrated	Clear audio; face mostly visible; most tasks shown	Audio unclear; face partially visible	Very poor quality	No video
B2	**1.5**	Verbally explains: (1) WHY CNN needs no flattening until after feature extraction — spatial structure preservation; (2) VALID vs SAME padding and the output size formula; (3) WHY MaxPooling passes only the max gradient and zeros all	Explains 5–6 concepts with reasoning	Demonstrates steps without theory	Reads from cells; no analysis	No verbal explanation

		others; (4) LeNet-5 historical significance and differences from modern CNNs; (5) WHY data augmentation reduces overfitting — effective dataset expansion; (6) Feature Extraction vs Fine-Tuning — what gets frozen, WHY fine-tuning needs a low lr; (7) Residual connections — WHY skip connections solve vanishing gradient in deep nets				
B3	**0.5**	5–10 min; all 6 task groups shown; feature map grid visible; training curves with annotations visible; results table shown	4–5 min; most tasks shown	2–4 min; key tasks missing	Under 2 min	Not submitted

Subtotal B: 1.0 + 1.5 + 0.5 = 3.0 Marks

Component C: GitHub Repository + README.md (2 Marks)

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
C1	**0.5**	Repo: DL_PR4A.ipynb + DL_PR4A.html + /plots (feature maps, augmentation grid, training curves, confusion matrix, results table) + requirements.txt	Key files present; a few plots missing	Only .ipynb; no plots	Repo exists but incomplete	No repo
C2	**1.0**	README: title + CIFAR-10 source + all 6 topics described (CNN, LeNet-5, Backprop,	Title + topics + partial plots	Title + dataset only	Blank or single line	No README

		Augmentation, Transfer Learning, Residual) + plots with captions + results table + video link + tools section				
C3	**0.5**	3+ commits: e.g. 'Add Task A1–A3: CIFAR-10 EDA, convolution demo, LeNet-5'; 'Train CNN with EarlyStopping and ReduceLROnPlateau'; 'Add MobileNetV2 fine-tuning and results comparison table'	2–3 adequate commits	1 bulk commit	Unclear messages	No commits

Subtotal C: 0.5 + 1.0 + 0.5 = 2.0 Marks

PROJECT B — TEACHABLE MACHINE: TRAIN, EXPORT & DEPLOY IN PYTHON

Field	Detail
Project	Project B — Teachable Machine: Export & Deploy in Python
Subject	Deep Learning · Practical Report 4 (PR 4)
Platform	teachablemachine.withgoogle.com (browser-based, no account required)
Exports Required	3 models: Image Project · Audio Project · Pose Project — each exported as TensorFlow SavedModel or Keras .h5
Python Files	3 deployment scripts + 1 combined application — saved as DL_PR4B/
Framework	TensorFlow 2.x + OpenCV + sounddevice / pyaudio (for audio)
Project Duration	2 Days · 2 Hours per Day = 4 Hours total
Total Marks	10 (Component A: 5 · Component B: 3 · Component C: 2)

§ 1B · What is Teachable Machine?

Teachable Machine (teachablemachine.withgoogle.com) is a free browser-based tool by Google that lets anyone train a real neural network model — no code required. It uses your webcam or microphone to collect training samples, trains a small model in the browser using TensorFlow.js, and exports it in multiple formats including TensorFlow SavedModel (.pb), Keras model (.h5), or TensorFlow Lite (.tflite). In this project you will use Teachable Machine to rapidly create three models, then write Python code to load and deploy each exported model — building something genuinely useful with each one.

Field	Detail
Website	https://teachablemachine.withgoogle.com
Image Project	Train on webcam photos or uploaded images — classifies image frames into custom categories
Audio Project	Train on short microphone recordings — classifies audio snippets into custom categories
Pose Project	Uses PoseNet to detect body keypoints — classifies body poses into custom categories
Export Format	Choose: TensorFlow → 'Keras' (.h5 + labels.txt) for easy Python loading; OR SavedModel folder for production
Required Python	pip install tensorflow opencv-python sounddevice numpy matplotlib
Audio extra	pip install sounddevice OR pip install pyaudio

No account needed	Training and export work without a Google account; models are trained and exported directly in your browser

§ 2B · Project B Tasks — What to Do

Complete ALL tasks. Each task = one Teachable Machine training session + one Python deployment script. All scripts go in the DL_PR4B/ folder. Document each model's class names, number of training samples per class, and training settings in your README.

Task B1: Image Project — Gesture / Object Classifier

#	Action	Details / Configuration
B1.1	**Choose your image classification problem**	Select one of the following (or propose your own with 4+ classes): (a) Hand gesture recognition — collect: thumbs up, thumbs down, peace sign, fist, open palm (5 classes). (b) Fruit classifier — collect: apple, banana, orange, no-fruit background (4 classes). (c) Face emotion classifier — collect: happy, sad, neutral, surprised (4 classes). Minimum: 4 classes, 80+ samples per class (Teachable Machine works best with 100–200 per class).
B1.2	**Collect training data in Teachable Machine**	Rename Class 1, Class 2, etc. to your actual class names. Use 'Webcam' to collect 80–120 samples per class by clicking 'Hold to Record'. Vary: lighting, background, angle, distance. After all classes are collected, click 'Train Model'. Training takes 1–3 minutes in the browser. Note the final accuracy shown in 'Preview'. Screenshot the Training panel showing class names + sample counts.
B1.3	**Export and verify the model**	Click 'Export Model' → TensorFlow tab → Keras → 'Download my model'. Extract the ZIP. You get: keras_model.h5 and labels.txt. Place both in DL_PR4B/models/image/. Open labels.txt — confirm class names are listed one per line, prefixed 0, 1, 2…
B1.4	**Deploy: image_classifier.py**	Write DL_PR4B/image_classifier.py: (a) Load model: model = tf.keras.models.load_model('models/image/keras_model.h5', compile=False). (b) Load labels: labels = open('models/image/labels.txt').read().splitlines(). (c) Open webcam: cap = cv2.VideoCapture(0). (d) Per frame: resize to 224×224, normalise to [-1,1]: img = (img / 127.5) - 1.0. Expand dims to (1,224,224,3). (e) prediction = model.predict(img, verbose=0). class_idx = np.argmax(prediction). confidence = prediction[0][class_idx]. (f) Overlay class name + confidence bar on frame. Press 'q' to quit. Save a screenshot to DL_PR4B/output/b1_image_result.jpg.
B1.5	**Build something useful: gesture-controlled tool**	Extend image_classifier.py to build one of: (a) Gesture volume control — map 'thumbs up' → pyautogui.press('volumeup'), 'thumbs down' → volumedown. (b) Gesture slideshow — map gestures to keyboard arrow keys for controlling a PowerPoint presentation. (c) Fruit freshness logger — detect fruit class and log the timestamp + class to a CSV file every 5 seconds. Add a Markdown comment block at the top of the extended script explaining the application, how to use it, and which gestures/objects it recognises.

Task B2: Audio Project — Sound / Command Classifier

#	Action	Details / Configuration
B2.1	**Choose your audio classification problem**	Select one: (a) Voice commands — collect: 'next', 'back', 'stop', 'start', background noise (5 classes). (b) Sound recognition — collect: clapping, snapping fingers, knocking, whistle, silence (5 classes). (c) Musical note detector — collect: 'do', 're', 'mi', 'fa', 'sol' (5 classes). Minimum: 4 classes, 50+ 1-second clips per class.
B2.2	**Collect audio data**	Teachable Machine records 1-second audio clips via your microphone. Important: record a 'Background Noise' class first (50+ clips of ambient silence/room noise) — this prevents false positives when no target sound is present. Collect 50–80 clips per class. Train the model (takes 1–2 min). Note the confusion matrix shown in Preview — which classes get confused? Screenshot the training panel.
B2.3	**Export audio model**	Export Model → TensorFlow → Keras → Download. Extract to DL_PR4B/models/audio/. Files: keras_model.h5, labels.txt, metadata.json. Note: Teachable Machine audio models expect a specific input format — 44,100 Hz sample rate, mono, 1-second clips converted to a mel-spectrogram internally. The exported model actually takes a (1, 43, 232, 1) spectrogram tensor as input — check metadata.json for confirmation.
B2.4	**Deploy: audio_classifier.py**	Write DL_PR4B/audio_classifier.py: (a) Load model and labels. (b) Record 1-second audio clips continuously: import sounddevice as sd. audio = sd.rec(int(44100), samplerate=44100, channels=1, dtype='float32'). sd.wait(). (c) Preprocess: the raw audio must be converted to the same spectrogram format Teachable Machine used internally. Use tensorflow.lite.support or the tm-audio-snippet approach: normalise audio to [-1,1], compute mel spectrogram using tensorflow_io or scipy. Alternatively, use the TFLite export + tflite_runtime for the full preprocessing pipeline. (d) Print predicted class + confidence every second. (e) Overlay predictions in a live terminal display using print with carriage return.
B2.5	**Build something useful: audio-triggered action**	Extend audio_classifier.py to: (a) Voice command controller — 'next' triggers cv2.imshow of the next image in a folder; 'stop' closes the window. (b) Clap counter — count clap events and display the running total in a terminal. (c) Sound alarm — trigger an alert (beep + red screen flash using OpenCV) when a specific sound (e.g., a whistle) is detected with > 85% confidence. Document the chosen application in a comment block.

Task B3: Pose Project — Body Pose Classifier

#	Action	Details / Configuration
B3.1	**Choose your pose classification problem**	Select one: (a) Yoga pose classifier — collect: mountain pose, warrior I, warrior II, child's pose, downward dog (5 poses). (b) Exercise counter — collect: squat down, squat up, rest, arm raised, arm down (5 states). (c) Sign language letters — collect: A, B, C, D, E (5 hand-shape letters). (d) Attentiveness detector — collect: attentive (looking forward), distracted (looking away), absent (no person). Minimum: 3 poses, 60+ samples per pose.
B3.2	**Collect pose data**	Teachable Machine Pose Project uses PoseNet to extract 17 body keypoints (nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles) from your webcam. It classifies based on the relative positions of these keypoints — not the raw image pixels. This makes it

		illumination-invariant. Collect 60–100 samples per pose. Vary: distance from camera, clothing colour. Train the model. Screenshot the Preview showing keypoint skeleton overlay.
B3.3	**Export pose model**	Export → TensorFlow → Keras → Download. Extract to DL_PR4B/models/pose/. The pose model takes a (1, 17, 2) tensor of normalised (x, y) keypoint coordinates — not an image — as input. To get keypoints from a webcam image in Python, you need PoseNet or MoveNet. Use: pip install tensorflow_hub. import tensorflow_hub as hub. movenet = hub.load('https://tfhub.dev/google/movenet/singlepose/lightning/4').
B3.4	**Deploy: pose_classifier.py**	Write DL_PR4B/pose_classifier.py: (a) Load MoveNet for keypoint extraction. (b) Load the exported Teachable Machine pose model. (c) Load labels.txt. (d) Per webcam frame: run MoveNet to get 17 keypoints (each: y, x, confidence). Flatten to (1,34) — 17 keypoints × 2 coordinates. Feed into the pose model. (e) Draw keypoints and skeleton lines on the frame using cv2.circle and cv2.line. (f) Overlay predicted pose class + confidence. (g) Save output video to output/b3_pose_result.mp4. Press 'q' to quit.
B3.5	**Build something useful: pose-triggered application**	Extend pose_classifier.py for one of: (a) Exercise rep counter — detect 'squat down' → 'squat up' transitions and increment a counter; display reps on screen. (b) Yoga coach — display the expected pose name + a reference image overlay; turn the frame border green when the detected pose matches the target pose with > 80% confidence. (c) Attentiveness alert — if 'distracted' is detected for > 3 consecutive seconds, play a beep sound (use winsound on Windows or os.system('afplay') on Mac). Document the application fully in comments.

Task B4: Combined Application — Teachable Machine Multi-Modal Demo

#	Action	Details / Configuration
B4.1	**Design the combined application**	Write a comment block at the top of DL_PR4B/combined_app.py describing: (a) What the application does. (b) Which three models are loaded (image/audio/pose). (c) What the user interaction flow is. Example concept: 'Smart Classroom Monitor' — Image model detects student presence; Audio model detects background noise level; Pose model detects attentiveness. Display a dashboard frame showing the status of all three simultaneously.
B4.2	**Build combined_app.py**	Load all three models at startup. Open webcam. In the main loop: (a) Run image classifier every frame. (b) Run pose classifier every frame (using MoveNet keypoints). (c) Run audio classifier on a background thread (non-blocking — use threading.Thread to record + classify audio every 2 seconds). (d) Compose a dashboard frame: split the webcam frame into regions using cv2.putText and cv2.rectangle panels — one status panel per model. (e) Display the combined dashboard with cv2.imshow. Record output to output/b4_combined.mp4.
B4.3	**Transfer Learning concept — Markdown**	Add a Markdown document (DL_PR4B/CONCEPTS.md): (1) What Teachable Machine does internally: it trains a small neural network on top of a pre-trained feature extractor (MobileNet for images, AudioNet for audio, PoseNet for pose) — this is Transfer Learning via Feature Extraction. (2) WHY it works: MobileNet was pre-trained on ImageNet; its convolutional feature extractor already knows edges, textures, and object parts. Teachable Machine only trains the final classification head on your custom classes. (3) Compare Teachable Machine's approach to the manual Feature Extraction in Project A, Task A6.2 — they are the same concept, one visual and one coded. (4) Limitation: Teachable Machine models are small and optimised for speed, not maximum

		accuracy — for production applications, fine-tuning a larger model (as in Task A6.4) would yield higher accuracy.
B4.4	**Model introspection**	Load each .h5 model in Python and print: model.summary(), model.input_shape, model.output_shape. Compare: How many layers does each Teachable Machine model have? How many trainable parameters? Add a comment in combined_app.py: Teachable Machine's image model is typically MobileNetV2 truncated at an intermediate layer + a small Dense head — only the Dense head weights are your custom-trained parameters.

§ 3B · Marking Scheme — Project B (10 Marks)

Total Marks: 10 — same 5+3+2 structure as Project A. Same rubric applies.

Component	Marks	Weight	Deliverable
A. Technical Completion (Scripts + Exported Models + Outputs)	5 Marks	50%	All 3 models exported + 3 deployment scripts + combined app running
B. Recorded Video Explanation (Face + Screen)	3 Marks	30%	5–10 min MP4/MOV: face + screen showing live demos of all 3 models + combined app
C. GitHub Repository + README.md	2 Marks	20%	Public repo with scripts, models, outputs, CONCEPTS.md, screenshots, video link
TOTAL	**10 Marks**	**100%**	

Component A: Technical Completion (5 Marks)

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
A1	**1.5**	Image model: trained in Teachable Machine (4+ classes, 80+ samples each, screenshot); exported keras_model.h5 + labels.txt; image_classifier .py runs on webcam with correct 224×224 normalisation to [-1,1]; class + confidence overlay; extended useful application (gesture control or equivalent) functional	Image model exported and deployed; webcam classifier running; useful extension partially functional	Image model exported; script loads model and classifies a static image; no webcam loop	Model exported but deployment script fails to load or predict	No image model
A2	**1.5**	Audio model: trained (4+ classes including background noise, 50+ clips each, screenshot); exported; audio_classifier.	Audio model exported; script records and classifies audio; live prediction printing; extension	Audio model exported; script loads model; no real-time recording loop	Model exported; script loads but audio recording or preprocessi ng fails	No audio model

		py records 1-sec clips, preprocesses, predicts, prints class+confidence continuously; extended audio-triggered application functional	partially functional			
A3	**1.0**	Pose model: trained (3+ poses, 60+ samples each, screenshot with keypoint skeleton visible); exported; pose_classifier.py uses MoveNet for keypoints; feeds (1,34) tensor to pose model; draws skeleton on frame; displays class+confidence; output video saved; extended application (rep counter or yoga coach) functional	Pose model exported; MoveNet running; keypoints drawn; classification displayed; no extended application	Pose model exported; script loads model; keypoints not drawn or classification incorrect	Pose model exported; MoveNet not integrated; placeholder script only	No pose model
A4	**1.0**	combined_app.py: all 3 models loaded at startup; audio running on background thread (non-blocking); image + pose running per frame; dashboard frame with 3 status panels; output video saved; CONCEPTS.md: Transfer Learning explanation linking Teachable Machine to manual Feature Extraction; model.summary() introspection for all 3 models	Combined app running with 2 of 3 models; dashboard displayed; CONCEPTS.md partially complete	Combined app loads models but only 1 model active; no dashboard	combined_app.py present but does not run	No combined app

		with layer count and parameter comparison				

Subtotal A: 1.5 + 1.5 + 1.0 + 1.0 = 5.0 Marks

Component B: Recorded Video — Face + Screen (3 Marks)

Record a video showing your face (webcam PiP) and screen simultaneously. Run each script live. Explain what Teachable Machine is doing internally.

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
B1	**1.0**	Clear audio and video; face visible via PiP; all live demos shown on screen (Teachable Machine training UI + all 3 Python scripts running live + combined app)	Clear audio; face mostly visible; most demos shown	Audio unclear; face partially visible	Very poor quality	No video
B2	**1.5**	Verbally explains: (1) What Teachable Machine does — trains a Dense classification head on top of a frozen pre-trained feature extractor in the browser; (2) WHY this is Transfer Learning — the base model (MobileNet/AudioNet/PoseNet) is pre-trained; only the head is custom-trained; (3) WHY preprocessing matters — image must be resized to 224×224 and normalised to [-1,1] to match training conditions; (4) HOW PoseNet/MoveNet works —	Explains 4–5 concepts with reasoning	Demonstrates without explaining	Reads from comments	No explanation

		extracts 17 keypoints from body; the pose model classifies keypoint positions, not pixels; (5) WHY a background noise class is needed for audio — prevents the model outputting a class when no target sound is present; (6) HOW threading prevents the audio recording loop from blocking the webcam frame loop				
B3	**0.5**	5–10 min; shows Teachable Machine training UI (at least 2 projects); shows all 3 Python scripts running live on webcam/mic; shows combined app dashboard	4–5 min; 2 models shown	2–4 min; 1 model shown	Under 2 min	Not submitted

Subtotal B: 1.0 + 1.5 + 0.5 = 3.0 Marks

Component C: GitHub Repository + README.md (2 Marks)

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
C1	**0.5**	Repo: DL_PR4B/ folder with all 4 scripts + DL_PR4B/models/ (3 × keras_model.h5 + labels.txt) + /output/ (3 demo videos/screenshots) + /screenshots/ (Teachable Machine training UI screenshots for all 3 models) + CONCEPTS.md + requirements.txt	All scripts and models present; a few outputs or screenshots missing	Scripts present; models not uploaded (link provided instead); no screenshots	Repo exists; only 1–2 scripts	No repo

C2	**1.0**	README: project title + 'Project B — Teachable Machine' declaration + setup instructions + description of all 3 trained models (problem, classes, sample counts) + CONCEPTS.md summary + screenshots of Teachable Machine UI + output screenshots + video link + tools used (TF, OpenCV, sounddevice, tensorflow_hub)	Title + setup + 2–3 model descriptions + some screenshots	Title + setup only	Blank or single line	No README
C3	**0.5**	3+ commits: e.g. 'Add image model — gesture classifier with webcam deployment and volume control'; 'Add audio model — voice command classifier with 5 classes and background noise'; 'Add pose model — yoga classifier with MoveNet keypoints and rep counter'	2–3 adequate commits	1 bulk commit	Unclear messages	No commits

Subtotal C: 0.5 + 1.0 + 0.5 = 2.0 Marks

§ 4 · Grade Bands & Video Requirements (Both Projects)

Grade Bands — applies to Project A and Project B equally

Score	Grade	Descriptor
9.0 – 10.0	A+ Outstanding	Project A: all 6 task groups functional; residual block built; fine-tuning with concatenated history; full 6-model results table. Project B: all 3 models trained, exported, deployed, and extended; combined app running with threading; CONCEPTS.md complete — portfolio-ready
8.0 – 8.9	A Excellent	Project A: CNN + augmentation + Transfer Learning (FE + FT) functional. Project B: all 3 models deployed + combined app with 2–3 models integrated
7.0 – 7.9	B+ Very Good	Project A: CNN + augmentation + Feature Extraction; no fine-tuning or residual block. Project B: 3 models deployed; combined app partially integrated
6.0 – 6.9	B Good	Project A: CNN + augmentation; no transfer learning. Project B: 2 models deployed; no combined app
5.0 – 5.9	C Satisfactory	Project A: CNN functional; no augmentation or transfer learning. Project B: 1 model deployed; significant gaps
Below 5.0	D Needs Improvement	Project A: LeNet-5 or CNN not running. Project B: models not deployed. Rework required

Video Requirements — Both Projects

Requirement	Detail
Format	MP4 or MOV, maximum 500 MB
Face visibility	Webcam picture-in-picture overlay visible throughout the full recording
Screen — Project A	Show Jupyter Notebook running: feature maps, training curves, confusion matrix, results table, fine-tuning history
Screen — Project B	Show Teachable Machine browser UI for at least 2 projects + all Python scripts running live on webcam/mic + combined app dashboard
Duration	5 to 10 minutes
Naming convention	DL_PR4A_YourName_GRID.mp4 OR DL_PR4B_YourName_GRID.mp4
Upload location	Google Drive (Anyone with link can view) or YouTube (Unlisted)
Link placement	Paste the video URL in your GitHub README.md under the Video section

§ 5 · Submission Checklists

Tick your chosen project's checklist. You only need to complete one checklist.

PROJECT A CHECKLIST — CNN from Scratch

- ☐ DL_PR4A.ipynb created; pip install tensorflow numpy matplotlib pandas confirmed
- ☐ requirements.txt: tensorflow>=2.12.0, scikit-learn, numpy, matplotlib, pandas
- ☐ CIFAR-10 loaded via keras.datasets; X shapes confirmed (50000,32,32,3) and (10000,32,32,3)
- ☐ 5×10 sample image grid plotted with class names as column headers
- ☐ Pixel histogram before AND after normalisation plotted
- ☐ Labels one-hot encoded; y_train_ohe shape (50000,10) confirmed
- ☐ CNN vs ANN Markdown: spatial structure, translation invariance, weight sharing, parameter comparison
- ☐ Manual convolution with scipy.signal.convolve2d on greyscale CIFAR image
- ☐ VALID vs SAME output shapes printed with size formula for both
- ☐ Stride=1 vs stride=2 output shapes compared
- ☐ 16 feature maps from single Conv2D layer displayed in 4×4 grid
- ☐ MaxPooling vs AveragePooling feature maps before/after shown (32×32 → 16×16)
- ☐ Flatten output shape before/after printed and explained
- ☐ LeNet-5 built exactly to specification; parameter count verified for 2+ layers
- ☐ Custom CNN (VGGNet-style, < 2M params) built; model.summary() printed
- ☐ Markdown: WHY stacking two conv layers before each pooling
- ☐ Backprop in CNN Markdown: forward pass, max-location switch, AveragePool gradient, Flatten gradient
- ☐ Model compiled: Adam + categorical_crossentropy + accuracy
- ☐ EarlyStopping(patience=10, restore_best_weights=True) callback added
- ☐ ReduceLROnPlateau(factor=0.5, patience=5, min_lr=1e-6) callback added
- ☐ Training curves plotted with stopping epoch (red dashed) and LR reduction epochs (grey dashed) annotated
- ☐ Confusion matrix (10×10 with class names) generated
- ☐ 10 misclassified samples displayed in 2×5 grid with true and predicted labels
- ☐ Data augmentation visualised: 9 random transforms of one image in 2×5 grid
- ☐ CNN retrained with ImageDataGenerator; comparison plot (no-aug vs aug, 4 lines)
- ☐ CNN retrained with Keras augmentation layers (RandomFlip, RandomRotation, RandomZoom, RandomTranslation)
- ☐ CIFAR-10 images resized to 96×96 for MobileNetV2
- ☐ MobileNetV2 Feature Extraction: base_model.trainable=False; trainable param count confirmed
- ☐ Feature Extraction trained 20 epochs; test accuracy recorded
- ☐ Fine-Tuning: top-30 layers unfrozen; recompiled at Adam(lr=1e-5); trained 10 more epochs
- ☐ Concatenated history plot with fine-tuning start line annotated
- ☐ Full 6-model results table with highlight_max on Test Accuracy
- ☐ Residual connection block built in Keras Functional API
- ☐ Restart & Run All — zero errors before submission
- ☐ Notebook exported as HTML (DL_PR4A.html)
- ☐ Video: face PiP + screen, 5–10 min, MP4/MOV, named DL_PR4A_YourName_GRID.mp4
- ☐ Video explains: CNN vs ANN, VALID/SAME, max-location switch, LeNet-5 history, augmentation, feature extraction vs fine-tuning, residual connections
- ☐ GitHub: public repo with DL_PR4A.ipynb + .html + /plots + requirements.txt
- ☐ README: CIFAR-10 + all 6 topics + plots + 6-model results table + video link
- ☐ 3+ meaningful commits with descriptive messages
- ☐ Video link in README under Video section

PROJECT B CHECKLIST — Teachable Machine

- ☐ DL_PR4B/ folder created with sub-folders: models/image/, models/audio/, models/pose/, output/, screenshots/
- ☐ pip install tensorflow opencv-python sounddevice tensorflow_hub numpy matplotlib
- ☐ requirements.txt committed
- ☐ Teachable Machine Image Project: 4+ classes, 80+ samples per class, trained in browser
- ☐ Screenshot of Teachable Machine Image training panel (class names + sample counts) saved to screenshots/
- ☐ keras_model.h5 + labels.txt exported to DL_PR4B/models/image/
- ☐ image_classifier.py: model loads; webcam opens; frame resized to 224×224; normalised to [-1,1]; prediction overlay working
- ☐ Extended useful image application functional (gesture control / fruit logger / etc.)
- ☐ Screenshot or video of image_classifier.py running saved to output/
- ☐ Teachable Machine Audio Project: 4+ classes including background noise, 50+ clips each, trained in browser
- ☐ Screenshot of Teachable Machine Audio training panel saved to screenshots/
- ☐ keras_model.h5 + labels.txt + metadata.json exported to DL_PR4B/models/audio/
- ☐ audio_classifier.py: records 1-sec clips; preprocesses; predicts; prints class+confidence continuously
- ☐ Extended audio-triggered application functional
- ☐ Teachable Machine Pose Project: 3+ poses, 60+ samples each, trained in browser
- ☐ Screenshot of Teachable Machine Pose training panel with keypoint skeleton visible saved to screenshots/
- ☐ keras_model.h5 + labels.txt exported to DL_PR4B/models/pose/
- ☐ pose_classifier.py: MoveNet loaded via tensorflow_hub; keypoints extracted per frame; (1,34) tensor fed to pose model; skeleton drawn on frame; class+confidence overlay
- ☐ Output video saved to output/b3_pose_result.mp4
- ☐ Extended pose application functional (rep counter / yoga coach / attentiveness alert)
- ☐ combined_app.py: all 3 models loaded at startup
- ☐ Audio running on background thread (threading.Thread) — non-blocking
- ☐ Dashboard frame with 3 status panels displayed via cv2.imshow
- ☐ Output video saved to output/b4_combined.mp4
- ☐ CONCEPTS.md: Transfer Learning explanation linking TM to manual Feature Extraction; model.summary() introspection for all 3 models
- ☐ Video: face PiP + screen, 5–10 min, MP4/MOV, named DL_PR4B_YourName_GRID.mp4
- ☐ Video shows Teachable Machine training UI (at least 2 projects) + all 3 Python scripts running live + combined app
- ☐ Video explains: TM as Transfer Learning, WHY preprocessing, PoseNet keypoints, background noise class, threading
- ☐ GitHub: public repo with DL_PR4B/ scripts + models + output + screenshots + CONCEPTS.md + requirements.txt
- ☐ README: 'Project B — Teachable Machine' declared + setup + 3 model descriptions with classes/sample counts + screenshots + video link
- ☐ 3+ meaningful commits with descriptive messages
- ☐ Video link in README under Video section

© 2026 Red & White Skill Education. All Rights Reserved.